Integrate a Modern Lazy-Loading Alternative for Large-Scale Spectroscopic Database Processing

Google Summer of Code 2026 | OpenAstronomy

Mentors: Nicolas Minesi @*minouhub*, pratham Hole @*Prtm2110*

1. About Me---------- 2
1.1 Basic Information---------- 2
1.2 Platform Details---------- 2
1.3 Background---------- 2
1.4 Contributions to Radis/Radis---------- 3
1.5 Interest In Open Astronomy---------- 4
2. Overview---------- 4
3. Goals---------- 4
4. Project Description---------- 5
4.1 Why Vaex Needs to Be Replaced---------- 5
4.2 Benchmarks---------- 6
4.2.1 File Size Comparison---------- 6
4.2.2 Memory Usage (Peak RAM)---------- 6
4.2.3 Full Benchmark Results---------- 6
4.2.4 Why Polars Over DuckDB?---------- 7
4.3 Migration Strategy---------- 7
4.4 I/O Layer Refactoring---------- 7
4.5 Computation Layer Refactoring---------- 8
4.6 Backward Compatibility---------- 8
5. Timeline---------- 9
6. Commitment to GSoC---------- 10
7. Why Me?---------- 10

1. About Me

1.1 Basic Information

- **Name** Zeyad Zahran
- **University** Zagazig University, Faculty of Computers and Informatics (2023 - Present)
- **Major** Computer Science
- **Email address** zeyadzahran06@gmail.com
- **Github** Zeyadzahran
- **LinkedIn** Zeyad Zahran
- **Slack** Zeyad Zahran
- **Contact Number** +20 1004 417 028
- **Timezone** Eastern European Standard Time (GMT+02:00), Egypt

1.2 Platform Details

- **OS** Fedora Linux 43
- **Editor** Visual Studio Code
- **CPU** Intel(R) Core(TM) i7-6820HQ (8) @ 3.60 GHz
- **Installed RAM** 16GB

1.3 Background

I am a third-year undergraduate student from Egypt, studying Computer Science at Zagazig University. As one who was raised in a family Obsessed of scientific so Physics, Mathematics, and science have always been fascinating topics for me.

My journey in programming started with competitive programming, which built the foundation of how I approach problems. I have solved **1000+ problems across different platforms**, with most of them on **Codeforces (my account)**, where I regularly participate in contests and practice algorithmic problem solving.

I also qualified for the **ICPC regional contest (Africa and Arab Collegiate Programming Championship – ACPC)**, where I worked on algorithmic challenges in a team setting under strict time constraints.

Outside competitive programming, I mainly work as a **backend developer**, building APIs and backend systems using technologies such as **Node.js, FastAPI, Laravel, and SQL/NoSQL databases**. I am also currently learning machine learning.

Some of the projects I built include:

- **Magitsh:** A simplified Git implementation from scratch built with Node.js .

- **Apple Seed:** A small 2D platformer game built with Unity and C# .
- **Hogwarts for Programming:** A PHP web application where users can enroll in courses, take quizzes, and interact with a gamified learning system that houses points and leaderboards.
- **Digital Library API:** A backend API for a digital library system built with Node.js, Express, and MongoDB.

Alongside personal projects, I started exploring **open source development** by reading different codebases and understanding how real systems are structured.

Recently, I contributed to the **Talawa project (Palisadoes Foundation),** where I worked on improving **test coverage and reliability**. My contributions include merged pull requests in both repositories:

- **Talawa Admin:** Test coverage improvements. [**MERGED**]
- **Talawa API:** Notification query test improvements. [**MERGED**]
- **Talawa Chat Logic:** Improved notification logic by preventing users from seeing their own messages as unread. *ISSUE*

1.4 Contributions to Radis

In a short time, I have worked on several issues and pull requests to help improve Radis. Here is a summary of my contributions:

1.4.1 Issues (All Issues)

1. Feature: Add Feather format support for cache files #919
2. Enhancment: Physical barwidth (`hwhm_voigt`) for Line Survey plots #929
3. [Benchmark] Extend `vaex_vs_pandas_performance.py` to include Polars/Parquet and file-parsing memory profiling #931
4. [BUG]: `planck()` and `planck_wn()` silently produce wrong results for T <= 0 #932
5. Profiling import radis startup time: Current status vs #629 #937
6. Bare `except:` blocks swallow KeyboardInterrupt and mask errors #945
7. Automated issue assignment via GitHub Actions #952
8. Bug: `raise DeprecationWarning` crashes the program instead of issuing a warning #963
9. Remove optically_thin option from Spectrum.rescale #972
10. Add copy=False to normalize spectrum #974

1.4.2 Pull Requests (All PRs)

NOTE: Please check all PRs link for the latest updates, as some might get merged or I might submit new ones after this proposal.

1. add validation for non-positive temperatures in planck functions #934 [**MERGED**]

2. perf: optimize startup time by lazy loading heavy dependencies #941 [**MERGED**]
3. fix: replace all bare except: with specific exception types #949 [**MERGED**]
4. ci: fix Travis coverage upload to radis/radis Codecov project #960 [**MERGED**]
5. feat: support barwidth='hwhm_voigt' in LineSurvey for physical linewidths #943 [Waiting for review]
6. ci: cache HITRAN databases to fix rate-limiting and mark network tests #951 [Waiting for review]
7. ci: add issue assignment and inactivity notice workflows #959 [Waiting for review]
8. fix: replace raise DeprecationWarning with warnings.warn and NotImplementedError #964 [**MERGED**]
9. refactor: extract envelope logic from gpu to standalone module #970 [Waiting for review]
10. refactor: deprecate optically_thin parameter #973 [Waiting for review]
11. perf: add copy=False to spectrum.get() in compare.py #975 [Approved by *@Prtm2110 waiting for @minouhub*]
12. fix: update pressure units and fix database loading bug #985 [Waiting for review]
13. test: optimize test_eq_vs_noneq_isotope to use local offline database #986 [Waiting for review]

1.5 Interest In Open Astronomy

- Since I was a kid, I have loved space. I spent a lot of time watching documentaries and following space news. Later, when I started studying computer science, my focus shifted to programming and problem-solving. While looking at open-source projects, I found RADIS through OpenAstronomy. I really liked how it connects programming with physics and math in a clear way. That is what made me interested in OpenAstronomy and motivated me to choose this project.

2. Overview

- **RADIS:** A high-resolution infrared molecular spectra code used for characterizing plasmas, flames, and atmospheres. To compute spectra, RADIS loads molecular databases (HITRAN, HITEMP, ExoMol, GEISA) and applies physics equations to calculate line strengths.
- **Database Scale:** Some databases are massive. HITEMP CO2 is over 50GB. To handle this without crashing, RADIS adopted Vaex for its memory-mapping and lazy-loading capabilities.
- **The Maintenance Issue:** Vaex is now unmaintained. It does not work on Python 3.12+ and is incompatible with NumPy 2.x. This blocks RADIS from upgrading to modern Python.

- **Proposed Solution:** This project will replace Vaex with a modern lazy-loading alternative. Based on my benchmarks, both **Polars** and **DuckDB** are strong candidates. I propose focusing on **Polars** as the primary choice because its DataFrame API is similar to Vaex, making migration easier. However, the final decision will be made with mentors after evaluating both options during the bonding period.

3. Goals

By the end of the coding period, I aim to accomplish the following:

- Replace all Vaex dependencies with a modern alternative (Polars or DuckDB) throughout the RADIS codebase
- Migrate cache format from HDF5 to Parquet with backward compatibility for existing files
- Ensure HITEMP CO2 (50GB+) loads with peak RAM under 4GB using lazy evaluation
- Maintain or improve calc_spectrum() computation speed compared to current implementation
- Achieve 100% test pass rate on Python 3.11, 3.12, and 3.13
- Document the migration path for existing users with clear upgrade instructions
- Have all code merged to main branch with passing CI/CD tests

4. Project Description

4.1 Why Vaex Needs to Be Replaced

Vaex was a good choice when it was adopted, but it is now a maintenance risk:

- **Unmaintained:** No releases since 2023, no active development
- **Broken:** Does not work on Python 3.12+ due to NumPy 2.x incompatibility
- **No future:** The project appears abandoned

RADIS cannot stay on Python 3.11 forever. The dependency must be replaced with something actively maintained.
Vaex is embedded in two main layers of the codebase:

I/O Layer handles reading and writing database files:

- `radis/api/hdf5.py` The `DataFileManager` class, central hub for all file operations
- `radis/api/dbmanager.py` `DatabaseManager` for loading multiple files
- `radis/api/exomolapi.py` ExoMol transition/state file parsing
- `radis/api/hitempapi.py` HITEMP chunked loading (the 50GB CO2 path)

Computation Layer handles spectrum calculation:

- `radis/lbl/base.py` Energy column assignments using `vaex.vconstant()`.
- `radis/lbl/loader.py` Database concatenation and sorting
- `radis/levels/partfunc.py` Partition function binning with `binby`

4.2 Benchmarks

I ran benchmarks on real CO2 HITRAN data (1,059,080 spectral lines) to validate the approach. I compared Pandas, Polars, Dask, and DuckDB across different file formats. Full benchmark code: [radis-gsoc-benchmarks]

4.2.1 File Size Comparison

Parquet compresses spectroscopic data extremely well **6.6x smaller** than HDF5:

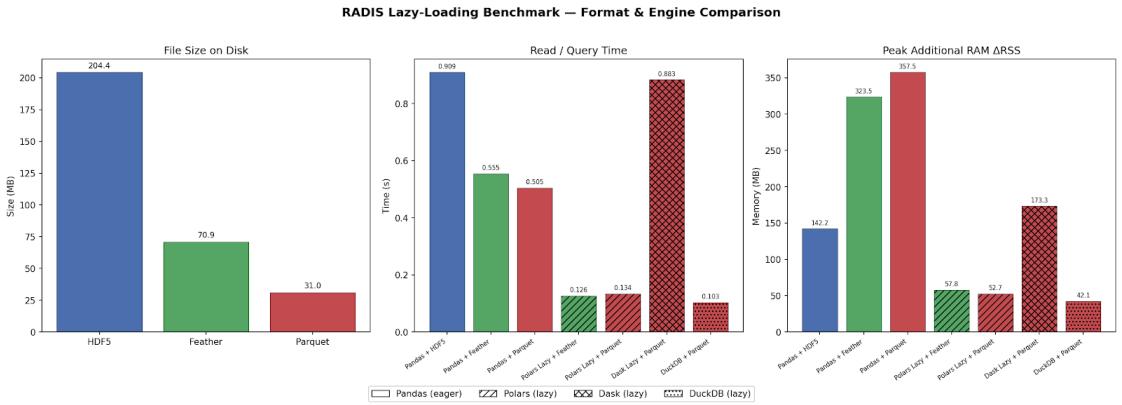
```
============================================================
  FILE SIZES
============================================================
    HDF5        :     204.36 MB
    Feather     :      70.87 MB
    Parquet     :      30.99 MB

============================================================
```

4.2.2 Memory Usage (Peak RAM)

This is the key metric. Each engine runs in its own subprocess to get accurate memory readings:

```
================================================================
  BENCHMARK RESULTS
================================================================
  Method                        Time (s)   Peak ΔRSS (MB)
  -------------------------------------------------------
  Pandas + HDF5                   0.9093          142.17
  Pandas + Feather                0.5546          323.52
  Pandas + Parquet                0.5046          357.52
  Polars Lazy + Feather           0.1261           57.75
  Polars Lazy + Parquet           0.1335           52.68
  Dask Lazy + Parquet             0.8833          173.35
  DuckDB + Parquet                0.1029           42.11
```

4.2.3 Full Benchmark Results



DuckDB + Parquet Fastest (0.09s) and lowest memory (42 MB)

Polars + Parquet Very fast (0.16s), low memory (53 MB), pure Python API

4.2.4 Why Polars Over DuckDB?

While both Polars and DuckDB successfully solve the memory bottleneck via lazy evaluation and Parquet filter pushdowns, DuckDB is slightly faster, but I recommend **Polars as the primary choice** for these reasons:

1. **Easier migration:** Polars has a DataFrame API similar to Vaex (`df.filter()`, `df.with_columns()`, `df.group_by()`). DuckDB uses SQL, which would require rewriting all queries.

2. **Same patterns:** Vaex code like `vaex.concat(frames)` maps directly to `pl.concat(frames)`. With DuckDB, we would need to register tables and write SQL strings.

3. **Pure Python:** Polars integrates naturally with NumPy arrays and existing RADIS code. No context switching between Python and SQL.

The performance difference (0.07s) is negligible for RADIS use cases where spectrum computation takes seconds. The migration effort saved by using Polars is more valuable.

That said, this is not a final decision. During the community bonding period, I will prototype both approaches with mentors and pick the best fit for RADIS.

4.3 Migration Strategy

Based on the benchmarks, I propose replacing Vaex with **Polars + Parquet**:

1. **Polars** replaces Vaex as the DataFrame engine - similar API makes migration straightforward
2. **Parquet** replaces HDF5 as the cache file format - 6.6x smaller, native lazy loading support

3. **Backward compatibility** old HDF5 files continue to work via pytables fallback

If during implementation DuckDB proves better for specific use cases (like complex aggregations), we can use it selectively while keeping Polars as the main engine.

The user-facing API does not change. Users still call `calc_spectrum()` the same way. The changes are internal to the I/O and computation layers.

API Mapping:

The migration is straightforward because Polars mirrors Vaex's API closely. Functions like `vaex.concat()` become `pl.concat()`, `vaex.open()` becomes `pl.scan_parquet()`, and `vaex.from_pandas()` becomes `pl.from_pandas()`. The only notable difference is replacing `vaex.vconstant()` with `pl.lit()` for constant columns, and switching from `binby` aggregations to Polars' `group_by().agg()` pattern.

4.4 I/O Layer Refactoring

To systematically replace the Vaex dependencies, the I/O layer refactoring will be executed across the following key modules:

1. **Core I/O Integration (`radis/api/hdf5.py`):** Add the Polars engine to `DataFileManager`. Implement Parquet-specific `read()`, `write()`, and `read_filter()` methods, ensuring metadata is preserved via Parquet schemas.

2. **Multi-File Loading (`radis/api/dbmanager.py`):** Enable lazy evaluation by replacing the legacy `vaex.open()` with `pl.scan_parquet()`.

3. **API Parsing Updates (`radis/api/exomolapi.py` & `radis/api/hitempapi.py`):** Swap `vaex.from_csv()` for `pl.scan_csv()` in ExoMol, and replace `vaex.concat()` in HITEMP's chunked CO2 loading (tackling the critical 50GB+ memory bottleneck).

4. **Cache System Updates (`radis/api/cache_files.py`):** Register "polars" as a valid engine and update internal logic to detect the new Parquet cache formats.

Example: HITEMP Chunked Loading (hitempapi.py):

```python
# BEFORE
df = vaex.concat([vaex.open(f) for f in chunk_files])

# AFTER
lf = pl.scan_parquet(chunk_files)  # Accepts list of files
df = lf.filter(pl.col("wav").is_between(wmin, wmax)).collect()
```

4.5 Computation Layer Refactoring

Following the I/O layer updates, the core computation layer will be refactored to utilize Polars' fast, vectorized operations across the following modules:

1. **Base Operations (`radis/lbl/base.py`):** Replace `vaex.vconstant()` with Polars' `pl.lit()` to create constant columns. Also, rewrite the `anynan_vaex()` function to use Polars' built-in null checking.

2. **Merging Isotopes (`radis/lbl/loader.py`):** Replace `vaex.concat()` with Polars' native concatenation. This will make merging and sorting different isotope datasets much faster and cleaner.

3. **Math & Grouping (`radis/levels/partfunc.py`):** Upgrade how data is grouped and calculated. Replace Vaex's legacy `binby` method with Polars' `group_by()` and `cut()` functions to boost performance.

Example: Energy Column Assignment (base.py):

```python
# BEFORE
if self.dataframe_type == "vaex":
    df["Evibl"] = vaex.vconstant(np.nan,
length=df.length_unfiltered())

# AFTER
if self.dataframe_type == "polars":
    df = df.with_columns(pl.lit(np.nan).alias("Evibl"))
```

Example: Partition Function Binning (partfunc.py):

```python
# BEFORE
result = df.mean("gj", binby="Erot", limits=[Emin, Emax],
shape=256)

# AFTER
bins = np.linspace(Emin, Emax, 257)
result = (
    df.with_columns(pl.col("Erot").cut(bins).alias("bin"))
    .group_by("bin")
```

```
    .agg(pl.col("gj").mean())
    .sort("bin")
)
```

4.6 Backward Compatibility

Users with existing HDF5 cache files will not be forced to re-download databases:

1. `DataFileManager.guess_engine()` detects format by file extension
2. Old .hdf5 files are read via pytables fallback
3. Optional auto-conversion to Parquet when `AUTO_UPDATE_DATABASE=true` in `radis.json`
4. Deprecation warnings guide users to convert gradually

Configuration Changes:

`radis/default_radis.json`:

```JSON
{
  "MEMORY_MAPPING_ENGINE": "auto",
  "DATAFRAME_ENGINE": "polars",
  "CACHE_FILE_FORMAT": "parquet"
}
```

`pyproject.toml`:

```None
[project.dependencies]
polars = ">=1.0.0"
pyarrow = ">=15.0.0"

[project.optional-dependencies]
legacy = ["vaex"]  # For users who need backward compatibility
```

5. Timeline

Community Bonding Period (May 1 - May 24)

- Set up development environment with Polars, DuckDB, PyArrow, pytest
- Prototype both Polars and DuckDB on a small RADIS module to compare migration effort

- Finalize the choice with mentors based on API compatibility and performance
- Study the Vaex code paths in detail
- Create a GitHub tracking issue listing all modules to modify
- Start a blog to document the journey

Week 1, 2 (May 25 - June 7)

- Finalize benchmarks on HITEMP H2O subsets
- Design adapter layer in `DataFileManager` for the chosen engine
- Implement `read()`, `write()`, `read_filter()` for Parquet

First Evaluation Deliverable: Benchmark report + working `DataFileManager`

Week 3, 4 (June 8 - June 21)

Finish `hdf5.py` refactoring

- Refactor `dbmanager.py`, `exomolapi.py`, `hitempapi.py`
- Implement LRU cache system for HITEMP CO2 chunks

Week 5, 6 (June 22 - July 5)

- Refactor `base.py` - replace `vaex.vconstant()` calls
- Refactor `loader.py` and `partfunc.py`
- Numerical validation: compare `calc_spectrum()` output between engines

Week 7, 8 (July 6 - July 19)

- Test HITEMP CO2 (50GB+), verify peak RAM < 4GB
- Handle backward compatibility with existing HDF5 files

Second Evaluation Deliverable: All Vaex code replaced, unit tests passing, HITEMP CO2 loads without crash

Week 9, 10 (July 20 - August 2)

- Update test files to cover the new engine
- Run full test suite on Python 3.11, 3.12, 3.13
- Add backward compatibility tests
- Numerical accuracy tests: pandas vs polars output

Week 11, 12 (August 3 - August 16)

- Update `default_radis.json` and `pyproject.toml`
- Write a migration guide for users
- Performance comparison report (before/after)
- Final code review and cleanup

Final Evaluation Deliverable: Vaex fully replaced, all tests green, documentation complete

6. Commitment to GSoC

- I will dedicate about 40 hours a week to GSoC and make it my top priority. I have a short exam period, but I will manage my time so it does not affect the project. I am already working on RADIS and will keep contributing before the coding period begins. During GSoC, I will share regular updates, communicate well with my mentors, and work extra hours if needed. I also plan to stay active in RADIS long after GSoC is over.

7. Why Me

- My competitive programming background taught me to break problems into smaller parts, try different approaches until I find a solution, and learn new things quickly without getting stuck easily.
- In a short time, I got familiar with the project. I read the RADIS documentation and explored the codebase to see how components interact.
- I have completed the Spectro101 tutorial to understand spectroscopy concepts.
- I didn't stop at exploring; I started making real changes. I opened issues, suggested improvements, and submitted pull requests with fixes, refactoring, and enhancements.
- I joined the RADIS Slack community and became part of the discussions there. I interacted with contributors and mentors, asked questions when needed, and shared ideas and suggestions.
- I participated in discussions and shared suggestions (relevant discussions: feature discussion 1, feature discussion 2, bugs & tests discussion)